Nazar Vision

Sea Turtle Hatchling Detection and Conservation Monitoring

Software Engineering Graduation Project Specification



Institution	Final International University, Northern Cyprus
Department	Software Engineering
Project	Nazar Vision
External technical mentor	Siraj Aizlewood, PhD (siraj.aizlewood@hotmail.com)
Start date	October 2026
Duration	One to two academic terms
Target prototype demonstration	May 2027
Project type	Software Engineering / Computer Vision / Machine Learning
Development language	Python

1. Project Description

Nazar Vision is a Python-based computer vision project focused initially on the automatic detection of sea turtle hatchlings in images and video footage.

A longer-term aim is to develop a wildlife monitoring system that can detect hatchlings as they emerge from their nests, potentially alerting conservation volunteers so they can respond when necessary. The system could

also be extended to identify adult turtles (e.g. when a female turtle comes to shore to lay eggs) and, ultimately, other wildlife species.

The graduation project will focus on developing and evaluating a working proof of concept, using publicly available annotated imagery to train an object detection model capable of recognising sea turtle hatchlings.

The project will make use of an existing Python codebase called *Nazar Vision*, which integrates established computer vision and machine learning tools. Students will work on the software's development, testing, integration, experimentation and evaluation.

The intention is not to develop an object detection algorithm from scratch, but to create a practical and reproducible workflow using existing technologies.

2. Background and Motivation

Sea turtles nest on beaches in Northern Cyprus and many other parts of the world. Conservation organisations monitor nesting activity, protect nests and observe hatchling emergence. This work often requires volunteers to be physically present, sometimes throughout the night, or to review recorded footage.

Computer vision could potentially support these activities by automatically identifying turtle activity and notifying conservation teams when relevant events occur. Modern object detection models, such as YOLO (You Only Look Once), can be trained to recognise and locate particular objects within images or video frames.

Although open-source wildlife detection technologies already exist, their suitability for specialised conservation applications remains limited in some important respects. For example, Microsoft's MegaDetector is widely used to identify animals in camera-trap imagery but does not distinguish between individual species. Other tools, such as Google's SpeciesNet, can classify animals at the species level. However, their reliability for detecting small sea turtle hatchlings under nighttime beach conditions is uncertain.

Furthermore, existing wildlife detection systems have documented limitations when dealing with very small animals and reptiles. MegaDetector's own documentation acknowledges difficulties detecting reptiles in earlier model versions, as well as challenges involving small animals and unusual camera perspectives. Although newer models have improved, this illustrates the limitations of relying exclusively on general-purpose wildlife detectors.

A key opportunity is to develop custom detection models trained specifically for the species, camera positions, lighting conditions and environments in which they will operate. For example, a model trained using footage from a fixed camera monitoring a particular nesting beach could potentially achieve substantially better performance than a general-purpose detector, particularly when identifying small hatchlings under challenging nighttime conditions. Such improvements would need to be demonstrated through evaluation.

However, preparing annotated datasets, training custom models, evaluating their performance and integrating them into monitoring systems typically requires specialist technical knowledge. This can present a barrier for conservation organisations with limited technical resources. **Nazar Vision aims to address this gap by providing an accessible framework with publicly available source code that enables conservation organisations to develop and refine their own species-specific detection models, using footage collected from their particular monitoring locations.**

Sea turtle hatchling detection provides an initial application with several technical challenges:

- Hatchlings are small and may occupy only a tiny proportion of a camera image.
- Emergence frequently occurs during darkness or low-light conditions.
- Sand, rocks, vegetation, shadows and other animals may be incorrectly identified as hatchlings.
- Camera positioning, resolution, lighting and environmental conditions vary between locations.
- A model trained on publicly available photographs may not perform reliably on footage captured at a real nesting site.
- The same hatchling may appear in many consecutive frames, potentially generating repeated detections and notifications.

The initial development of Nazar Vision focuses on preparing annotated datasets, training an object detection model and evaluating its performance. Later development may introduce object tracking, statistical confidence estimation, automatic notifications and integration with suitable camera-monitoring systems.

A central long-term objective is to allow conservation organisations to train or refine models using their own site-specific footage, rather than relying exclusively on general-purpose, one-size-fits-all detectors. By making this process more accessible, Nazar Vision aims to help conservation organisations develop detection systems tailored to their own species, camera environments and monitoring requirements, potentially achieving significantly better performance in their intended operating conditions. Therefore, once sufficiently refined and tested, the software is intended to be made publicly available with its source code, such that conservation organisations and researchers around the world can freely use, adapt and improve.

3. Project Objectives

3.1 Core Objectives

The core project shall:

1. Provide a functioning workflow for importing and reviewing annotated object detection datasets.
2. Allow users to inspect and correct annotations through FiftyOne.
3. Prepare suitable training data for a YOLO object detection model.
4. Train a model capable of identifying sea turtle hatchlings.
5. Evaluate the model using recognised object detection metrics.
6. Provide visual inspection of model predictions and detection errors.
7. Demonstrate object detection on prerecorded video footage.
8. Produce documented, tested and reproducible software.

These objectives form the required proof of concept.

3.2 Extension Objectives

Depending on the duration of the project and progress against the core requirements, students may also:

1. Incorporate additional publicly available datasets to improve model performance.
2. Extend the annotation and training workflow to support an `adult_turtle` class, allowing the model to detect and distinguish between hatchlings and adult sea turtles.
3. Investigate methods for improving the detection of small hatchlings.

4. Implement object tracking to identify and follow individual turtles across consecutive video frames.
5. Use model confidence scores to estimate how likely each detection is to be correct, and combine evidence across video frames to improve that estimate.
6. Develop a system that generates alerts when potential hatchlings or adult turtles are detected.
7. Export trained models for use with existing video-monitoring software.
8. Test detection, tracking and alerts using prerecorded footage as a simulated live video feed.
9. Investigate and test suitable nighttime camera equipment, including connecting a physical camera to the detection system and assessing its performance under controlled low-light conditions.
10. Demonstrate how the framework could be adapted to detect other wildlife species.

These are additional objectives rather than compulsory components of the core project. Hatchling detection remains the priority, with adult turtle detection and the other extensions undertaken where time and resources permit.

4. Existing Software and Technical Environment

4.1 Nazar Vision Codebase

Nazar Vision is an existing Python project under active development. The project already has an initial repository structure, development environment, configuration files, documentation and supporting code for dataset preparation and object detection workflows.

The students will receive access to the repository and relevant project documentation. The first development activity will be to review the existing implementation and determine which functionality is complete, incomplete or requires improvement. Student contributions will be identified accordingly, ensuring that the assigned work represents meaningful software engineering development.

4.2 Technologies

The intended technologies are:

Technology	**Purpose**
Python	Primary development language
FiftyOne	Dataset inspection, annotation management and model evaluation
Ultralytics YOLO	Object detection training, inference and tracking
PyTorch	Underlying deep learning framework used by Ultralytics
OpenCV	Optional image processing, video handling and camera integration
Git / GitHub	Version control and collaborative development
uv	Python package and environment management
pytest	Automated testing
Ruff	Linting and code quality checks
Black	Code formatting

Technology	Purpose
mypy	Static type checking
scikit-learn	Optional statistical analysis and confidence calibration
ByteTrack / BoT-SORT	Optional multi-object tracking
Frigate or comparable software	Potential video monitoring and deployment integration

The initial object detection model is expected to use a small pretrained Ultralytics YOLO model, currently YOLO26s. Students may investigate alternative model sizes or configurations if justified by experimental results and available computing resources.

4.3 Design Principles

The software should:

- Use existing open-source libraries wherever suitable.
- Avoid duplicating functionality already provided by FiftyOne, Ultralytics or other established tools.
- Follow established software engineering principles and best practices where appropriate, including DRY (Don't Repeat Yourself), KISS (Keep It Simple, Stupid), separation of concerns and modular design.
- Prioritise readable, maintainable and well-structured code, avoiding unnecessary complexity or premature abstraction.
- Maintain a clear distinction between original data, reviewed annotations and model predictions.
- Support reproducible model training and evaluation.
- Preserve the original source datasets.
- Use configuration files for dataset-specific mappings and training parameters.
- Design the software to accommodate additional datasets, models and detection tasks without extensive modifications.
- Include appropriate automated tests, static type checking, linting, consistent formatting and documentation.

The students should prioritise a reliable, understandable implementation over unnecessary complexity.

5. Available Datasets

5.1 Initial Dataset

The initial dataset is 'Sea Turtles Hatchling Detection', published by IFPE on Roboflow Universe.

Property	Details
Dataset	Sea Turtles Hatchling Detection
Version	24
Images	694
Licence	CC BY 4.0

Property	Details
Export date	1 October 2026
Annotation format	YOLO object detection

The dataset includes existing training, validation and test partitions and two source classes:

- `sea turtle`
- `non-sea-turtle`

The initial Nazar Vision training task uses a single positive class, `hatchling`, with the following mapping:

Source label	Training label
`sea turtle`	`hatchling`
`non-sea-turtle`	Excluded from positive annotations

Excluding a `non-sea-turtle` annotation does not exclude the image itself. Images without positive hatchling annotations may remain as background examples. Entire images that are unsuitable for training or evaluation should instead be assigned the `excluded` sample tag in FiftyOne.

Dataset quality and augmentation

The downloaded dataset includes augmented images generated using horizontal and vertical flips, brightness adjustments, Gaussian blur and salt-and-pepper noise. Roboflow reports creating three versions of each source image.

Students must review the original annotations for missing or incorrect labels, particularly adult turtles labelled `sea turtle` that would otherwise be incorrectly mapped to hatchling. They must also investigate possible data leakage between training, validation and test partitions, including whether augmented versions of the same source image appear across different splits. Such overlap could produce misleadingly high evaluation results. Any identified issues should be corrected through the FiftyOne dataset review workflow before relying on model evaluation results.

5.2 Dataset Review

The source dataset should be inspected for:

- Annotation accuracy and completeness.
- Missing or incorrectly positioned bounding boxes.
- Duplicate and near-duplicate images.
- Potential overlap between training and evaluation data (data leakage).
- Unusable or unsuitable images.
- Consistency of class definitions.
- Suitability for training a hatchling detector.

Students should document significant dataset issues and any corrective action taken. Unsuitable images should be excluded by removing their `train`, `val` or `test` tag and assigning the `excluded` sample tag instead. This keeps the images available in FiftyOne without including them in training or evaluation.

The software must ensure that all changes made within FiftyOne, including annotation corrections, changes to dataset split assignments and image exclusions, are correctly persisted in MongoDB. A sensible backup strategy must also be implemented for the MongoDB database, with periodic backups or checkpoints that allow the working dataset, including its reviewed annotations and split assignments, to be restored if necessary. The original source image files must remain unchanged.

5.3 Additional Data

Additional publicly available datasets may be used during an extension phase. Any additional data should be checked for:

- Relevance to hatchling detection.
- Annotation quality.
- Licensing and permitted usage.
- Potential duplication of existing data.
- Compatibility with the training workflow.

Footage from local conservation organisations would be particularly valuable but is not currently guaranteed. Therefore, the core project must be achievable using the available public dataset.

6. System Workflow

The intended workflow is:

1. Import annotated images and labels into FiftyOne, preserving the original annotations and split assignments.
2. Create `ground_truth_reviewed` as an editable copy of the original annotations.
3. Review and correct annotations, modify `train`, `val` and `test` sample tags, and mark unsuitable images with the `excluded` tag.
4. Investigate data-quality issues, duplicates and possible leakage. These review activities can happen in any order.
5. Automatically regenerate canonical training labels from the latest reviewed annotations when preparing a dataset export.
6. Validate the annotations and sample tags, then export only images assigned to `train`, `val` or `test` for YOLO training.
7. Train an object detection model.
8. Evaluate the model using validation data during development, reserving the held-out test set for final assessment.
9. Import predictions into FiftyOne for visual analysis.
10. Review errors and improve the data or training configuration.
11. Retrain and compare results where appropriate.
12. Run inference on images and prerecorded video.
13. Save the trained model for later use, with optional export to a compatible monitoring system.

Dataset review activities do not need to follow a rigid sequence. Annotation corrections, split changes and data-quality investigations should be supported within the same FiftyOne working environment.

The workflow could also be extended to track individual turtles across video frames, use repeated detections to improve confidence that an object is a turtle, and send alerts when turtles are detected.

7. Core Functional Requirements

The following Functional Requirements (FRs) define the minimum functionality expected from the project. Some functionality may already exist in the Nazar Vision codebase. Students should review the existing implementation, test it and complete or improve it where necessary.

The aim is to produce a working, reliable and reproducible system, making use of existing FiftyOne and Ultralytics functionality wherever possible.

FR-01: Dataset Import

The system must support importing an annotated object detection dataset into FiftyOne.

It must:

- Import the source images and annotations.
- Preserve the original annotation data.
- Import existing train, validation and test assignments where available.
- Make images, annotations and split assignments accessible through FiftyOne.
- Avoid modifying the original source dataset files.

Acceptance criteria

- The initial Roboflow dataset can be imported successfully.
- Images, bounding boxes and labels are visible in FiftyOne.
- Original split assignments are imported correctly.
- Original source files remain unchanged.

FR-02: Dataset Review and Editing

The system must use FiftyOne to allow users to inspect and correct annotations, review dataset quality, and modify training, validation and test assignments.

How the dataset is organised

When the original Roboflow dataset is imported into FiftyOne, its annotations are stored in `ground_truth_source`, and its existing train, validation and test assignments are stored as sample tags (`train`, `val`, `test`). A copy of the original annotations is then created in `ground_truth_reviewed`, allowing users to correct them without changing the originals. FiftyOne's Annotation Schema must be configured to allow users to edit bounding boxes and class labels in `ground_truth_reviewed` directly through the FiftyOne App, while keeping `ground_truth_source` read-only. The dataset therefore contains:

Field / Property	Purpose
`ground_truth_source`	Original imported annotations, preserved unchanged.

Field / Property	Purpose
`ground_truth_reviewed`	Working annotations that users can correct, add to or remove.
Sample tags (`train`, `val`, `test`, `excluded`)	Current dataset assignments. Images tagged `excluded` remain in FiftyOne but are omitted from training-data exports.
`ground_truth_canonical`	Training-ready annotations automatically generated later from `ground_truth_reviewed`, using the configured class mappings.

These are fields and properties of the same FiftyOne dataset, not separate datasets or workflow stages.

Each image should have exactly one of the following assignment tags: `train`, `val`, `test` or `excluded`. Images with no assignment tag are considered unassigned, while images with conflicting assignment tags must be corrected before export.

Reviewing and editing the dataset

Once the dataset has been imported and `ground_truth_reviewed` has been created, users should be able to perform the following activities within the same FiftyOne environment:

- Inspect images, bounding boxes and class labels.
- Correct incorrectly positioned bounding boxes in `ground_truth_reviewed`.
- Add annotations for hatchlings that have been missed.
- Remove incorrect annotations.
- Correct class assignments.
- Review and change training, validation and test assignments by editing the `train`, `val` and `test` sample tags.
- Exclude unsuitable images by removing their existing split tag and assigning `excluded`. The images and their annotations must remain available in FiftyOne, without modifying the original source files.
- Review duplicate and near-duplicate images.
- Investigate possible data leakage between training and evaluation datasets.

These activities can be performed in any order, as part of the same dataset review process. For example, a user might correct several bounding boxes, discover duplicate images, move some images from training to validation, and then continue correcting annotations.

Annotation corrections are saved in `ground_truth_reviewed`, while split assignments and image exclusions are managed using the sample tags (`train`, `val`, `test`, `excluded`). All changes to both the annotations and sample tags are persisted in FiftyOne's MongoDB database. The original annotations in `ground_truth_source` and the original image files must remain unchanged.

Existing FiftyOne functionality should be used wherever possible, rather than implementing separate annotation-editing or split-management interfaces in Nazar Vision.

Preparing the reviewed dataset for training

When the user is ready to train a model, Nazar Vision must:

1. Auto-generate `ground_truth_canonical` from the current `ground_truth_reviewed` annotations, applying the configured class mappings (FR-03).

2. Validate the current sample tags and training-ready annotations (FR-04).
3. Export the prepared dataset for YOLO training.

If the user subsequently corrects annotations or changes split assignments, these changes must be incorporated the next time the dataset is prepared for training.

Acceptance criteria

- Users can inspect and correct annotations in `ground_truth_reviewed` through FiftyOne.
- Users can change train, validation and test assignments by editing sample tags in FiftyOne.
- Annotation corrections, split changes and data-quality reviews can be performed in any order.
- Users can exclude unsuitable images by removing their split tag and assigning `excluded`, without modifying the original source files.
- Excluded images remain accessible in FiftyOne but are omitted from training-data exports.
- All changes are saved in MongoDB and remain available when the dataset is reopened.
- Original annotations and image files remain unchanged.
- The current reviewed annotations and split assignments are used when preparing data for training.

FR-03: Canonical Training Labels

The system must generate training-ready annotations from `ground_truth_reviewed` using configurable class mappings.

For the initial dataset, the intended mapping is:

Reviewed label	Training label
`sea turtle`	`hatchling`
`non-sea-turtle`	Excluded

The mapping must be stored in a project configuration file.

For example:

```
label_fields:
  source: ground_truth_source
  reviewed: ground_truth_reviewed
  canonical: ground_truth_canonical

canonical_mapping:
  sea turtle: hatchling
  non-sea-turtle: null
```

Nazar Vision must use this configuration to generate `ground_truth_canonical` from `ground_truth_reviewed`, using FiftyOne's existing functionality. For example, an annotation labelled `sea turtle` in `ground_truth_reviewed` becomes `hatchling` in `ground_truth_canonical`. An annotation labelled `non-sea-turtle` is excluded from the canonical annotations. This removes the annotation, not the image containing it.

The mapping must also support additional classes. For example, the optional adult turtle detection extension (EXT-02) would allow users to add `adult_turtle` annotations in `ground_truth_reviewed` and include `adult_turtle: adult_turtle` in the configuration. Any adult turtles originally labelled `sea turtle` must first be correctly relabelled in `ground_truth_reviewed` to avoid mapping them to `hatchling`.

Images without positive annotations may remain as background examples if assigned to `train`, `val` or `test`. Images tagged `excluded` are omitted entirely from training-data exports, regardless of their annotations.

When canonical labels are generated

`ground_truth_canonical` is a derived field that can be regenerated from `ground_truth_reviewed` at any time. It is not continuously synchronised with annotation changes.

The intended behaviour is:

1. **During dataset review:** Users freely edit `ground_truth_reviewed` and change sample tags in FiftyOne. These changes do not automatically trigger canonical regeneration.
2. **On explicit request:** Users may regenerate `ground_truth_canonical` when they want to inspect the current training labels in FiftyOne.
3. **During training-data export:** Nazar Vision must automatically regenerate `ground_truth_canonical` from the latest `ground_truth_reviewed` annotations using the configured class mappings. No separate user action is required.
4. **During model training:** The training workflow must use the freshly exported dataset, including the current class mappings and split assignments.

This means `ground_truth_canonical` may temporarily contain outdated annotations while the user is reviewing the dataset. This is expected and does not require continuous updates after every edit. Every training-data export must automatically regenerate the canonical annotations before they are used. Regeneration should use a small, repeatable function built around FiftyOne rather than a separate annotation-conversion system. The original source files and reviewed annotations must remain unchanged during canonical generation. Unexpected class labels must be reported rather than silently converted or discarded.

Acceptance criteria

- The class mapping is read from the project configuration.
- `ground_truth_canonical` is generated from `ground_truth_reviewed`.
- `sea turtle` annotations are converted to `hatchling`.
- `non-sea-turtle` annotations are excluded.
- Images without positive annotations are handled correctly.
- Editing `ground_truth_reviewed` does not require immediate canonical regeneration.
- Users can explicitly regenerate canonical annotations when needed.
- Every training-data export automatically regenerates the canonical field using the latest reviewed annotations and configured mappings.
- The exported dataset uses the current train, validation and test assignments, excluding images tagged `excluded`.
- Source and reviewed annotations remain unchanged during canonical generation.
- Unexpected labels are detected and reported.
- If EXT-02 is implemented, the same mapping and export workflow supports both `hatchling` and `adult_turtle` annotations.

FR-04: Dataset Splits and Validation

Before exporting data for YOLO training, Nazar Vision must validate the current dataset in FiftyOne, including its split assignments, exclusions and canonical annotations. Split assignments are managed directly through FiftyOne sample tags, as described in FR-02. Nazar Vision must use these existing assignments rather than maintain a separate system for editing them.

Acceptance criteria

- Every image must have exactly one assignment tag: `train`, `val`, `test` or `excluded`.
- Missing or conflicting assignment tags must be detected and reported. Export must be prevented until these issues are resolved.
- Images tagged `excluded` must remain accessible in FiftyOne but be omitted from all training-data exports.
- Canonical annotations must be validated, including checking for unexpected class labels and invalid bounding boxes. Invalid annotations must be reported and resolved before export.
- The exported training, validation and test datasets must reflect the current sample tags in FiftyOne, including any changes made during dataset review.
- Students must be able to investigate duplicate and near-duplicate images, particularly where related images appear across different dataset splits.
- Where source information is available, students should also investigate whether frames from the same video, camera sequence or nesting event have been assigned to different splits.
- Identified data leakage should be corrected where possible, and any unresolved limitations documented.

Dataset checks should use FiftyOne's existing functionality wherever appropriate.

FR-05: Data Persistence and Backups

All changes made to the working dataset in FiftyOne must be correctly saved in its MongoDB database. The working FiftyOne dataset must be configured as persistent (dataset.persistent = True) to preserve its contents between sessions.

This includes:

- Added, removed or corrected annotations.
- Changes to bounding-box positions.
- Changes to class labels.
- Changes to image exclusion status, including the addition or removal of `excluded` tags and associated split-tag changes.
- Changes to train, validation and test assignments.
- Generated canonical annotations.
- Other relevant dataset metadata.

The original source images and annotation files must remain unchanged.

MongoDB backups

A sensible backup strategy must be implemented for the MongoDB database, with periodic checkpoints, particularly after significant annotation corrections and before important training experiments. These must allow the working dataset to be restored, including its reviewed annotations, split assignments and associated metadata, without requiring a separate backup for every individual edit.

FiftyOne normally stores references to image files rather than storing the image contents directly in MongoDB. The backup process must therefore ensure that the corresponding source images remain available and that their paths can be resolved when the database is restored. Because the original image files are not being modified, they do not need to be duplicated with every MongoDB checkpoint.

Acceptance criteria

- Annotation corrections and split changes remain available after closing and reopening FiftyOne.
- MongoDB backups or dataset checkpoints can be created manually on request and are automatically created before exporting data for training or evaluation.
- Backups are not required after every individual edit, but should also be created periodically following significant annotation changes.
- A backup can be restored successfully in a test.
- The restored dataset contains the expected annotations, train/validation/test assignments, `excluded` tags and other relevant metadata.
- The restored dataset can access its associated image files.
- Original source image and annotation files remain unchanged.

FR-06: Model Training

The system must support training or fine-tuning a YOLO object detection model using the prepared dataset. The training process follows dataset review and preparation:

1. Read the current reviewed annotations and split assignments from FiftyOne.
2. Generate/regenerate `ground_truth_canonical` using the configured class mapping.
3. Validate the canonical annotations and current splits.
4. Export only images with valid `train`, `val` or `test` assignments in a format supported by Ultralytics. Images tagged `excluded` must not be exported.
5. Train or fine-tune the chosen YOLO model.
6. Save the model and training results.

The training workflow must:

- Use the selected training and validation partitions.
- Use annotations from `ground_truth_canonical`.
- Support configurable training parameters.
- Save trained model weights.
- Record relevant training configuration and results.
- Allow trained models to be loaded for inference and evaluation.

The initial model is expected to be YOLO26s, although other model sizes or configurations may be investigated. Training should use Ultralytics' existing functionality rather than implementing the underlying neural network or training algorithms independently. Students should record sufficient information to

reproduce important training experiments, including the dataset state, split assignments, model configuration and training settings.

Acceptance criteria

- The canonical labels are regenerated before training-data export.
- The exported dataset uses the current validated split assignments and omits all images tagged `excluded`.
- A YOLO model can be trained successfully.
- Model weights are saved and can be reloaded.
- Relevant training settings and results are recorded.
- The trained model can perform inference on previously unseen images.

FR-07: Model Evaluation

The system must support quantitative and visual evaluation of trained models. Evaluation should include recognised object detection metrics, such as:

- Precision.
- Recall.
- Mean average precision (mAP).
- False positives.
- False negatives.

Students should also investigate performance on smaller hatchlings and particularly difficult images where practical.

Evaluating predictions in FiftyOne

After training, the model must be run on appropriate held-out evaluation data. The resulting predictions should be imported into FiftyOne and stored separately from the original, reviewed and canonical annotations. For example, predictions from a training experiment could be stored in a field such as `predictions_run_001`. This allows users to compare model predictions with `ground_truth_canonical` without changing the reviewed annotations.

Students should examine examples of:

- Correctly detected hatchlings.
- Hatchlings missed by the model.
- Objects incorrectly identified as hatchlings.
- Incorrectly positioned bounding boxes.
- Difficult backgrounds, lighting conditions or camera perspectives.

If annotation errors are discovered during evaluation, they should be corrected in `ground_truth_reviewed`, followed by regeneration of the canonical field before further training or evaluation.

Repeated experiments

Students may improve the model by changing training settings, correcting annotations or adding suitable data. Where multiple models are trained, results should be compared using a consistent evaluation method.

Training decisions should normally be based on training and validation data, with the held-out test set reserved for independent assessment.

Acceptance criteria

- Predictions can be generated on held-out images.
- Quantitative evaluation results can be produced.
- Predictions can be imported into FiftyOne.
- Predictions can be compared visually against canonical ground-truth annotations.
- Representative correct and incorrect detections are documented.
- Model performance and its limitations are reported.
- Results from different experiments can be compared where appropriate.

FR-08: Video Inference

The system must support running a trained model on prerecorded video footage or sequences of images. The inference workflow must:

- Read and process video frames.
- Detect potential hatchlings.
- Produce annotated output showing bounding boxes and confidence scores.
- Allow users to review the resulting predictions or annotated video.

Where appropriate, detection results should also be saved in a machine-readable format for later analysis. If suitable genuine video footage is unavailable, a sequence of images may be used to demonstrate frame-by-frame inference, provided this limitation is clearly stated.

Object tracking, alerts and physical camera integration are not required for this core functionality. The core requirement is to demonstrate that a trained model can recognise potential hatchlings in individual video frames.

Acceptance criteria

- A prerecorded video or image sequence can be processed.
- Annotated output can be generated and viewed.
- Detected hatchlings and their confidence scores are displayed.
- The trained model can be loaded and used without retraining it for each video.

FR-09: Testing and Documentation

The software must include sufficient tests and documentation for another developer to understand, operate and reproduce the workflow. The project should use its established development tools, including:

- pytest for automated testing.
- Ruff for linting.
- Black for code formatting.
- mypy for static type checking.

Students should follow established software engineering principles where appropriate, including DRY, KISS, separation of concerns and modular design. The software should remain readable and maintainable, without

unnecessary abstractions or duplicated functionality.

Automated testing

Tests should cover important processing functionality, particularly:

- Dataset import and validation.
- Annotation-field creation and handling.
- Canonical label generation.
- Split assignment and exclusion-tag validation, including unassigned images, conflicting tags and correct exclusion during export.
- Dataset export.
- Persistence of working annotations and split changes.
- Backup and restore procedures.
- Other important functionality introduced during development.

Documentation

The documentation must explain:

- How to install and configure the project.
- How to import a dataset into FiftyOne.
- How to create and use the reviewed annotation field.
- How to correct annotations, change split assignments and mark unsuitable images with the `excluded` tag in FiftyOne.
- How canonical training labels are generated.
- How the dataset is validated and exported.
- How to train a model.
- How to evaluate predictions in FiftyOne.
- How to run inference on images and video.
- How MongoDB backups are created and restored.
- Known limitations and unresolved issues.

Important experimental configurations and results must be recorded so that model training and evaluation can be reproduced.

Acceptance criteria

- The documented workflow can be followed in an agreed supported environment.
- Automated tests can be executed.
- Ruff, Black and mypy checks can be run.
- The backup and restore procedure is documented and tested.
- Another developer can reproduce the main dataset preparation, training and evaluation workflow using the required data and computing resources.

8. Extension Requirements

The following requirements are optional. They may be undertaken during a second academic term or after the core system is complete, depending on available time, computing resources and student interests. Students are not expected to complete every extension. The core project focuses on detecting sea turtle hatchlings.

ID	Extension
EXT-01	Additional datasets
EXT-02	Detecting adult sea turtles
EXT-03	Improving small-object detection
EXT-04	Tracking individual turtles
EXT-05	Estimating detection correctness using confidence scores and repeated observations
EXT-06	Automatic detection alerts
EXT-07	Model export and monitoring software integration
EXT-08	Simulating a live video feed
EXT-09	Testing a physical nighttime camera
EXT-10	Detecting other wildlife species

EXT-01: Additional Datasets

Students may investigate whether adding more annotated sea turtle images improves the model's performance. Additional datasets could contain hatchlings, adult sea turtles or both, depending on which detection classes are being developed.

This may involve:

- Identifying additional publicly available datasets.
- Checking dataset licensing and permitted usage.
- Reviewing annotation quality.
- Importing additional data into FiftyOne.
- Mapping different source labels into a consistent training format.
- Checking for duplicate images and potential data leakage.
- Retraining the model using additional data.
- Comparing performance with the original model.

Additional datasets should use the same general FiftyOne workflow as the initial Roboflow dataset. Original annotations must be preserved in `ground_truth_source`, while corrections and any additional annotations are made in `ground_truth_reviewed`. Training, validation, test and exclusion assignments must continue to be managed using FiftyOne sample tags. Canonical labels must be automatically regenerated from the reviewed annotations before training-data export, using the configured class mappings. The aim is to establish whether the additional data genuinely improves detection, rather than assuming that a larger dataset automatically produces better results.

Acceptance criteria

- Train and evaluate a model using additional suitable data.
- Compare its performance with the original model using a consistent evaluation method.

EXT-02: Detecting Adult Sea Turtles

An important possible extension is to expand Nazar Vision from detecting only sea turtle hatchlings to detecting both hatchlings and adult sea turtles.

This would allow the system to identify two different types of turtle activity:

- Hatchlings emerging from nests and moving across the beach.
- Adult sea turtles arriving on beaches, potentially to nest.

The aim would be to develop a model capable of distinguishing between the two life stages rather than treating all sea turtles as a single class. This could help alert users when, for example, an adult female turtle comes to shore to lay eggs.

Adding an adult turtle annotation

The initial model uses one positive training class:

- `hatchling`

The extended model would use two positive training classes:

- `hatchling`
- `adult_turtle`

Users must be able to add and correct `adult_turtle` annotations in `ground_truth_reviewed`, using the same FiftyOne annotation-editing workflow already established for hatchlings. For example, if an image contains an adult turtle incorrectly labelled `sea turtle`, the user could correct its class to `adult_turtle` in `ground_truth_reviewed`, leaving `ground_truth_source` unchanged. Additional images containing adult turtles could also be imported and annotated. The class mapping configuration would then be extended to recognise the new class.

For example:

Reviewed label	Canonical training label
`sea turtle`	`hatchling`
`adult_turtle`	`adult_turtle`
`non-sea-turtle`	Excluded from positive annotations

The existing mapping from `sea turtle` to `hatchling` assumes that the reviewed annotation has been confirmed to represent a hatchling. Adult turtles must not be automatically mapped to `hatchling` simply because their original source label is `sea turtle`. Such annotations must be corrected during dataset review.

Where the life stage cannot be reliably determined, students should investigate the image further or exclude the uncertain example from the relevant training and evaluation data.

Using the existing workflow

Adding adult turtle detection should not require a separate annotation-management system.

The same workflow must support both classes:

1. Import images and original annotations into FiftyOne.
2. Review annotations in `ground_truth_reviewed`.
3. Correct hatchling annotations and add or correct `adult_turtle` annotations.
4. Review the current train, validation, test and exclusion tags.
5. Automatically regenerate `ground_truth_canonical` using the updated class mapping when the dataset is exported.
6. Train a model using the resulting annotations.
7. Evaluate detections for both classes in FiftyOne.

An image may contain hatchlings, adult turtles or both. The annotation workflow should support multiple objects and classes within the same image. The existing `ground_truth_source`, `ground_truth_reviewed` and `ground_truth_canonical` fields must continue to serve the same purposes.

Model training

Where sufficient labelled data is available, students may investigate training a single YOLO model that detects both classes. The model would produce bounding boxes identifying objects as either `hatchling` or `adult_turtle`. For example, an annotated image could contain:

- Two bounding boxes labelled `hatchling`.
- One bounding box labelled `adult_turtle`.

A single two-class model would be preferable where it performs sufficiently well, although students may compare this approach with separate models if justified by the available data or experimental results.

Evaluation

The extended detector must be evaluated separately for both classes. Students should investigate:

- Precision and recall for hatchlings.
- Precision and recall for adult turtles.
- Mean average precision across the detection classes.
- Adult turtles incorrectly identified as hatchlings.
- Hatchlings incorrectly identified as adult turtles.
- Other objects incorrectly identified as turtles.
- Whether adding the adult class improves or reduces hatchling detection performance.

The availability and quality of adult turtle training data will be important. If the initial dataset contains too few suitable adult examples, students may need to identify additional public datasets before attempting a two-class model. The system should not be assumed to identify a turtle's sex or determine whether nesting is occurring merely because an adult turtle has been detected.

Acceptance criteria

- Extend the existing FiftyOne annotation and canonical-label workflow to support an `adult_turtle` class and, where sufficient suitable data is available, train and evaluate a model capable of

distinguishing adult turtles from hatchlings.
- The evaluation must report performance for both classes and document any limitations caused by the available data.

EXT-03: Improving Small-Object Detection

Students may investigate methods for improving the detection of small hatchlings. Detecting hatchlings is particularly challenging because they may occupy only a small proportion of a camera image.

Possible experiments include:

- Starting with high resolution images and reducing resolution to compare performance.
- Comparing different YOLO model sizes.
- Adjusting training parameters.
- Investigating data augmentation.
- Using image tiling or other methods intended to improve small-object detection.
- Testing different inference settings.

Changes should be evaluated through controlled experiments.

If adult turtle detection has also been implemented, students should investigate whether changes intended to improve hatchling detection affect the performance of the adult class.

Acceptance criterion

Test at least one alternative approach and compare its performance against a documented baseline.

EXT-04: Tracking Individual Turtles

The system may be extended to track individual turtles across consecutive video frames. Instead of treating every detection as a separate animal, the system should attempt to recognise when the same turtle appears in multiple frames. For example, if three hatchlings move across a beach, the system should attempt to maintain a separate identifier for each hatchling. If adult turtle detection has been implemented, the same tracking functionality could also be used to follow adult turtles moving across a beach.

Existing tracking algorithms such as ByteTrack or BoT-SORT may be used through Ultralytics. The tracking system should, where possible:

- Assign a unique identifier to each tracked object.
- Associate detections of the same turtle across frames.
- Distinguish multiple turtles.
- Record how long an object has been tracked.
- Record the number of frames in which an object was detected.
- Produce annotated video showing tracking identifiers and detected classes.

Students should investigate difficulties such as objects disappearing temporarily, animals overlapping, tracking identifiers switching between animals and inconsistent class predictions.

Acceptance criteria

- Demonstrate object tracking on suitable prerecorded footage, with identifiers that remain reasonably consistent across frames. Document any observed tracking failures.
- Where adult detection has been implemented, tracking of adult turtles may also be demonstrated.

EXT-05: Estimating Detection Correctness

YOLO produces a confidence score for each detection. However, a confidence score of 90% does not necessarily mean that the detection has a 90% probability of being correct. Students may investigate how to use evaluation results to estimate the relationship between a model's confidence score and the actual frequency of correct detections. For example, if detections with confidence scores between 80% and 90% are correct 85% of the time on representative evaluation data, this information could help estimate how trustworthy similar future detections are. This process is known as confidence calibration.

Students may use existing statistical tools such as scikit-learn to investigate methods including logistic calibration or isotonic regression.

If the model detects both hatchlings and adult turtles, the reliability of confidence scores should be considered separately for each class where sufficient labelled data is available. A model might, for example, produce reliable confidence estimates for adult turtles but less reliable estimates for small hatchlings.

Combining evidence across video frames

If object tracking has been implemented, the system may also investigate whether repeated observations of the same animal provide a more reliable estimate of whether it is genuinely a turtle. For example, a hatchling might be detected in 50 consecutive frames, with each detection receiving a relatively high confidence score. Thus, repeated observations may provide stronger evidence than a single detection.

However, consecutive video frames are highly correlated. An object incorrectly identified as a turtle in one frame may be incorrectly identified again in many subsequent frames. Individual detection probabilities must therefore not simply be multiplied together or treated as independent observations. Students could investigate using information such as:

- Detection confidence across the track.
- Number of successful detections.
- Track duration.
- Consistency of detections.
- Movement characteristics.
- Variation in appearance across frames.

Statistical methods, potentially including Bayesian approaches, could be investigated where suitable labelled video data is available. The aim is to estimate how likely it is that a tracked object genuinely represents the detected class. For a hatchling detector, this means estimating whether the object is genuinely a hatchling. If adult detection is included, the same approach could be investigated for adult turtles. Estimated probabilities must be validated using independent labelled data, and any improvements assessed for statistical significance.

Acceptance criteria

- Evaluate a method for estimating detection correctness and, if suitable tracked video data is available, investigate whether combining observations across a track improves those estimates.

- Students should compare the results against a simpler baseline and document any statistical assumptions or limitations. A reliable model of how probability relates to confidence is not guaranteed, particularly if sufficient labelled video footage is unavailable.

EXT-06: Automatic Detection Alerts

The system may be extended to generate alerts when potential turtle activity is detected. For the initial hatchling detector, this would involve alerts when potential hatchlings appear. If adult turtle detection has also been implemented, the system could generate separate alerts for adult turtle activity.

A basic demonstration could:

1. Process video frames.
2. Detect a potential turtle.
3. Identify its predicted class, such as `hatchling` or `adult_turtle`.
4. Check whether a configurable detection rule has been satisfied.
5. Generate an alert.
6. Save an image or short video segment associated with the event.

The initial alert rule could use a detection-confidence threshold. Rules should also be put in place for how often alerts should be sent on each subsequent detection of a turtle within a certain timeframe, to avoid overwhelming users with multiple alerts.

If object tracking is available, the system could also consider:

- How many times the animal has been detected.
- How long it has been tracked.
- Whether it has been detected consistently.

If reliable track-level probability estimates have been developed, these could also be used when deciding whether to issue an alert. For example, the system could investigate generating an alert once the estimated probability of a genuine hatchling exceeds a configurable threshold - e.g. 99%. Note that waiting too long for very high confidence may also delay alerts or cause genuine activity to be missed.

Different alerts for hatchlings and adults

If both classes are supported, students may investigate different alert rules for hatchlings and adult turtles. For example:

- A hatchling detection could generate an alert indicating potential hatchling emergence.
- An adult turtle detection could generate an alert indicating that an adult turtle has been observed on the beach. These alerts would represent potential turtle activity, not confirmation of nesting or any other specific behaviour. Different thresholds or notification rules may be appropriate for the two classes, depending on their detection performance and conservation requirements.

Acceptance criteria

- Demonstrate a working alert triggered by a configurable detection rule, with an associated image or video segment available for review.
- Configure alerts to avoid sending them too frequently.

- If both classes are supported, demonstrate that alerts can identify the detected class.
- If tracking is implemented, investigate whether it helps reduce repeated or false alerts.

EXT-07: Model Export and Monitoring Software Integration

The trained model may be exported for use with existing video-monitoring or inference software. Possible technologies include:

- ONNX.
- TensorRT.
- Frigate.
- Other compatible monitoring or inference systems.

Students should investigate how an exported model could be connected to an existing monitoring application rather than developing an entirely new video-monitoring system. This may require investigating:

- Supported model formats.
- Input and output formats.
- Hardware requirements.
- Inference performance.
- Detection events and notifications.
- Support for multiple detection classes, if adult turtle detection is implemented.

Compatibility should be verified rather than assumed. Exporting a YOLO model does not guarantee that it can be used directly by every monitoring application.

Acceptance criteria

- Export a trained model and demonstrate that it can detect turtles using compatible inference software, such as ONNX Runtime or TensorRT.
- Document any compatibility issues or additional requirements.

EXT-08: Simulating a Live Video Feed

Students may test the detection system using prerecorded footage played back as though it were arriving from a live camera. This is different from processing an entire video file offline: Frames should be passed to the detection system progressively, at a controlled rate, to test how it behaves when video is continuously arriving.

The test could include:

- Processing incoming frames.
- Displaying detections as the video plays.
- Measuring processing speed.
- Tracking animals, if implemented.
- Triggering alerts, if implemented.
- Checking whether repeated detections produce unnecessary alerts.

This would help test how the software components behave together in a continuous monitoring system.

Acceptance criterion

Demonstrate the software processing prerecorded footage as a simulated live stream, showing detections and any implemented tracking or alert behaviour.

EXT-09: Testing a Physical Nighttime Camera

Students may investigate and test a physical camera suitable for eventual nighttime turtle monitoring. An initial exploratory budget of approximately US$100–200 is envisaged. The investigation should consider:

- Low-light image quality.
- Resolution.
- Field of view.
- Recording frame rate.
- Video-stream compatibility.
- Power requirements.
- Storage requirements.
- Weather resistance.
- Equipment cost.

The aim would be to test whether the camera can provide a usable video feed and whether the software can process it. Controlled testing could involve:

- Connecting the camera through USB, Ethernet or another supported interface.
- Receiving and displaying the video.
- Measuring image quality under low-light conditions.
- Processing the video using the trained model.
- Testing the software using a mock scene containing artificial hatchlings or other representative objects.

Such experiments would test camera integration and video-processing capabilities. They would not establish that the model can reliably detect real hatchlings or adult turtles on a beach at night.

Camera equipment must not introduce lighting that could disturb turtles. Some cameras (described as providing night vision) may use infrared illumination, which would need appropriate conservation assessment before field use. Any actual installation at a nesting site would require suitable permissions and collaboration with conservation specialists.

Acceptance criteria

- Document the capabilities, limitations, cost and potential suitability of at least one camera configuration, including any practical testing completed.
- If a physical camera is available, demonstrate that its video feed can be received and processed by the software.

EXT-10: Detecting Other Wildlife Species

Although Nazar Vision initially focuses on sea turtles, the framework might be generalized to support other wildlife detection tasks. The addition of adult turtle detection would demonstrate how the same workflow can support more than one detection class. A further extension could investigate whether the workflow can be applied to a different animal or group of animals.

This would involve:

- Importing another annotated dataset.
- Reviewing and preparing annotations.
- Configuring the appropriate classes.
- Training a detection model.
- Evaluating its performance.

The objective is not to develop a single model capable of recognising every species. Instead, the same framework should support training different models for different conservation applications.

Acceptance criterion

Demonstrate that Nazar Vision can be configured and used with another suitable annotated dataset without substantial changes to the core software.

9. Performance Evaluation

Evaluating model performance is an important part of the project. A model that produces convincing annotated images is not necessarily reliable. Students must measure performance using appropriate evaluation data and explain the model's limitations.

The core evaluation focuses on hatchling detection. If adult turtle detection is implemented, the evaluation must also assess the additional class.

9.1 Detection Performance

The initial hatchling detector should be evaluated using appropriate metrics, including:

- Precision.
- Recall.
- Mean average precision (mAP).
- False positives.
- False negatives.

Students should visually examine examples of correct and incorrect detections in FiftyOne. Where possible, they should investigate whether performance varies with object size, image quality, lighting and background complexity.

9.2 Dataset Leakage

Students must investigate whether closely related or duplicate images appear across training and evaluation datasets.

Examples include:

- Identical images in both training and test datasets.
- Near-duplicate images produced through augmentation.
- The same image but with a different orientation, or otherwise altered, between splits.
- Consecutive frames from the same source video appearing across different splits.

Such overlap can make model performance appear better than it really is.

Any identified leakage should be addressed where possible and documented. Split assignments should be corrected in FiftyOne, with the resulting changes saved in MongoDB. Subsequent training experiments must use the updated split assignments.

9.3 Reproducibility of Experiments

Important training experiments should be reproducible. All annotation corrections, sample tag changes and other dataset modifications made in FiftyOne must be persisted in MongoDB, with periodic backups to ensure the reviewed data can be recovered.

For each significant experiment, students should preserve:

- The dataset state, including reviewed annotations and train, validation, test and exclusion assignments.
- The canonical label mapping.
- The model and training configurations.
- The trained model and evaluation results.

A MongoDB checkpoint or training-data export should be retained for significant experiments, allowing the exact dataset used to be recovered even if further changes are subsequently made in FiftyOne. The corresponding source images must also remain available.

Note: If adult turtle detection is investigated, hatchling-only and two-class experiments should be recorded so their results can be compared fairly.

9.4 Confidence Estimation

If confidence calibration is investigated, students should compare raw model confidence scores with how often detections are actually correct. If both hatchling and adult turtle classes are supported, class-specific calibration may be investigated. If track-level confidence estimation is attempted, students must consider the correlation between consecutive video frames. Probability estimates should not be presented as reliable unless supported by appropriate evaluation.

9.5 Tracking and Alert Performance

If tracking or alerts are implemented, useful additional measurements include:

- How consistently objects retain the same tracking ID.
- How often tracking IDs change incorrectly.
- How often the same object triggers repeated alerts.
- False alerts per hour of video.
- Missed turtle-detection events.
- Time between a turtle first appearing and an alert being generated.

Where multiple detection classes are supported, students should investigate whether tracking and alert performance differ between hatchlings and adult turtles.

These measurements would help assess whether tracking and alerting improve the system's usefulness.

9.6 Performance Targets

No fixed detection-accuracy percentage is required for the initial proof of concept. The performance achieved will depend on the quality and suitability of the available data. The expected outcome is a functioning system with credible evaluation results and clearly documented limitations.

Successful performance on public datasets does not establish that the system is reliable for real-world nighttime monitoring.

10. Project Deliverables

10.1 Core Deliverables - One-Term Project

Deliverable	Description
D1 - Working software	Functioning Nazar Vision workflow for dataset preparation, YOLO training and evaluation
D2 - Dataset persistence and backups	Verified saving of annotations and sample tags in FiftyOne/MongoDB, with a tested backup and restore procedure
D3 - Trained detector	At least one trained hatchling detection model with saved weights and configuration
D4 - Evaluation results	Quantitative performance results and analysis of representative detection errors
D5 - Video demonstration	Annotated video or image-sequence output demonstrating hatchling detection
D6 - Source code and tests	Version-controlled code and automated tests for important functionality
D7 - Technical documentation	Instructions for installation, dataset preparation, training, evaluation, inference and database recovery
D8 - Academic submission	Final technical report and presentation according to university requirements

10.2 Extension Deliverables - Two-Term Project

Additional deliverables will depend on which optional extensions from Section 8 are undertaken. These may include improved detection models, adult turtle detection, object tracking, confidence estimation, automated alerts, model export, simulated live-video processing or physical camera testing. Each selected extension should be accompanied by appropriate experimental results, tests and documentation.

The specific extensions should be agreed at the beginning of the relevant development period.

11. Project Timeline

The project may run for one or two academic terms, depending on the university's assignment. The following schedule is provisional and may be adjusted to match the university's academic requirements and the work assigned to students.

Period	Main activities	Milestone
October 2026	Review existing Nazar Vision code, confirm student responsibilities, inspect the initial dataset and set up the development environment	Agreed development plan
November 2026	Dataset import, annotation review, split and exclusion changes, quality checks, MongoDB persistence and canonical label preparation	Working dataset preparation workflow
December 2026	YOLO training, model evaluation in FiftyOne, prediction review and video inference	Working hatchling detection proof of concept
January 2027	Testing, backup and restore verification, documentation and academic submission	Core project completed
February–March 2027	Selected extensions, potentially including additional datasets, adult turtle detection, model improvements and object tracking	Improved prototype
April–May 2027	Selected extensions, potentially including confidence estimation, alerts, simulated live video and physical camera testing	Target demonstration for conservation organisations
June 2027	Final testing, documentation and academic submission	Two-term project completed

11.1 May 2027 Demonstration

A target milestone is to demonstrate Nazar Vision to local conservation organisations by May 2027, ahead of the summer turtle monitoring season. The demonstration should ideally include:

- A functioning hatchling detector.
- Annotated images and video.
- Quantitative model evaluation results.
- A demonstration of the FiftyOne dataset review and Ultralytics training workflow.
- An explanation of the system's current limitations.

Depending on development progress, the demonstration may also include:

- Detection of both hatchlings and adult sea turtles.
- Tracking individual turtles.
- Simulated detection-triggered notifications.
- Separate notifications for hatchling and adult turtle detections.
- Results from confidence estimation experiments.
- Continuous video processing using a simulated live feed.
- Results from controlled testing with a physical camera.

The purpose of the demonstration is to show what has been developed, obtain feedback and explore possible collaboration with conservation organisations. The demonstration does not need to establish that Nazar Vision is ready for deployment on a real beach.

If the student project ends after one term, the completed prototype may be demonstrated or developed further by the project proposer or other collaborators independently of the students' academic requirements.

12. Project Success Criteria

The core project will be considered successful when:

1. Users can import, review and correct annotations in FiftyOne while preserving the original dataset.
2. Users can manage training, validation, test and exclusion assignments, with all changes correctly persisted in MongoDB and recoverable through backups.
3. Training-ready annotations are automatically generated from the latest reviewed annotations using configurable class mappings.
4. A YOLO model can be trained and evaluated using the prepared dataset, with predictions available for visual inspection in FiftyOne.
5. The trained model can detect hatchlings in prerecorded video or image sequences.
6. The software is tested, documented and sufficiently reproducible for another developer to use.

The detailed acceptance criteria are defined in Section 7.

Optional extensions will be assessed against their individual requirements in Section 8.

Note: Reliable nighttime detection at a real nesting site, operational alerts and field deployment are not required for completion of the graduation project.

13. Project Constraints

The following constraints should be considered when planning and carrying out the project:

- **Available data:** The core project must be achievable using the publicly available dataset. Additional datasets and footage from conservation organisations are not guaranteed.
- **Computing resources:** Model training and experimentation must be appropriate for the GPU and computing resources available to students.
- **Project scope:** Hatchling detection takes priority. Adult turtle detection and other extensions are optional and depend on available time and resources.
- **Real-world performance:** Successful evaluation on public datasets does not establish reliable detection under actual nighttime beach conditions.
- **Equipment and field testing:** Physical camera testing is optional. Any future beach deployment would require suitable equipment, permissions and conservation guidance.
- **Licensing:** Dataset and software licences must be respected, particularly if the project is subsequently released publicly.

14. Project Resources and Responsibilities

14.1 Students

Students will be responsible for reviewing the existing Nazar Vision codebase, implementing and testing assigned functionality, preparing datasets, training and evaluating models, and producing the required documentation and academic submissions.

Development work should use Git and established software engineering practices.

14.2 External Technical Mentor

Siraj Aizlewood, PhD

The external mentor initiated Nazar Vision and will provide access to the existing codebase and documentation. He will offer voluntary technical guidance, subject to availability, including support with FiftyOne, Ultralytics, software architecture, implementation approaches and evaluation methods.

Students remain responsible for their assigned work and academic deliverables.

14.3 University

Final International University will provide academic supervision, student assignment and assessment according to the department's graduation-project requirements.

The scope of individual student contributions will be agreed following the initial code review.

14.4 Conservation Organisations

Collaboration with local conservation organisations will be explored, particularly for feedback, access to representative footage and potential future field testing. Their participation is not required for the core project.

14.5 Equipment and Software

The project will use the existing Nazar Vision repository, the initial public dataset and the software described in Section 4.

Students will require suitable development computers and access to appropriate computing resources for model training. GPU availability is to be confirmed.

Additional adult turtle datasets, physical cameras and conservation-site footage are optional resources.

If camera testing is undertaken, an exploratory budget of approximately US$100–200 is envisaged, subject to equipment availability and funding.

15. Longer-Term Development

Nazar Vision is intended to develop beyond the initial graduation project into an accessible wildlife detection framework with publicly available source code. The following activities describe possible future development and are not compulsory student deliverables.

15.1 Conservation Field Testing

A longer-term objective is to collaborate with conservation organisations to test Nazar Vision at real sea turtle nesting sites. This could involve installing suitable nighttime cameras to detect hatchlings emerging from nests and adult turtles visiting nesting beaches.

Any field testing would require appropriate equipment, permissions, conservation guidance and evaluation under representative conditions. The initial objective would be a supervised field trial rather than immediate operational deployment.

15.2 Site-Specific Models and Continued Improvement

A central aim of Nazar Vision is to allow conservation organisations to train models using footage from their own monitoring locations. The existing FiftyOne and YOLO workflow could be used to review annotations, prepare datasets, train models and evaluate their performance. Models could then be improved over time by incorporating additional field footage, correcting detection errors and retraining.

The framework should support different detection classes, including hatchlings, adult turtles and potentially other wildlife species, without requiring substantial changes to the underlying software.

Future development may also include object tracking, more reliable confidence estimates, automated notifications and integration with existing camera-monitoring systems.

15.3 Public Release and Further Collaboration

Nazar Vision is intended to be made publicly available with its source code once sufficiently refined, tested and documented.

The framework is intended for wildlife conservation and research involving non-human animals. The project proposer wishes to prohibit its use for human surveillance, law enforcement, military operations or espionage. Before releasing the software, a suitable licence will be chosen to reflect these restrictions, while also considering the licences of third-party software and any intellectual property rights relating to student contributions.

The graduation project may also provide opportunities for future university collaboration, including additional student projects, internships and research into wildlife detection.

A public release and operational field deployment are not required outcomes of the initial graduation project.

16. Project Completion

The expected outcome is a working, tested and evaluated proof of concept demonstrating sea turtle hatchling detection using FiftyOne, Ultralytics YOLO and the Nazar Vision Python framework.

The preferred target is to demonstrate the prototype to local conservation organisations by May 2027, with the possibility of exploring future collaboration, additional development and eventual field testing.

Adult turtle detection and other extensions may be included where time and resources permit, but are not required for successful completion of the core project.